Dual discriminator GAN with self-attention

Harsh Goel (251110034)

Kishan Kumar Mishra (251110044)

Singampalli Sri Varshith (251110074)

Thoram Venkata Madhu Sai Kiran (251110082)

ABSTRACT

In this project, we implement a dual-discriminator GAN with self-attention. Our generator and discriminator architectures are based on DCGAN [Deep Convolutional GAN – Radford et al., 2015], providing stable convolutional layers for image synthesis. The dual-discriminator framework, which encourages diversity and reduces mode collapse, is adopted from D2GAN [Double Discriminator GAN – Nguyen et al., 2017]. To improve the capture of long-range dependencies in generated images, we integrate self-attention layers as proposed in SAGAN [Self Attention GAN – Zhang et al., 2019]. This hybrid approach allows us to leverage proven architectural stability, enforce diverse outputs, and improve global coherence in images.

Baseline Paper

Our baseline generator and discriminator architectures follow the DCGAN framework, first introduced in "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks" by Alec Radford. Although the paper is only available through arXiv, it has accumulated more than 20,000 citations on Google Scholar, making it a landmark work in the field.

The DCGAN paper introduced the architecture but it did not report Inception Score (IS) or Fréchet Inception Distance (FID). There are two main reasons for this: first, these metrics were proposed later (IS in 2016 and FID in 2017), and second, the DCGAN paper primarily focused on using the architecture for unsupervised representation learning rather than quantitative generative evaluation.

Generative Adversarial Networks (GANs): A Brief Overview

Introduction

Generative Adversarial Networks (GANs) represent a breakthrough in generative modelling that enables machines to produce novel, realistic data by learning from existing examples. Unlike discriminative models that focus on classification or regression, GANs learn to *generate* samples (images, audio, text) that resemble a target distribution. This capability has impacted many domains including art, gaming, healthcare, and data augmentation.

GAN Architecture

A GAN comprises two components trained simultaneously:

Generator (G). The generator is a neural network that maps a noise vector $z \sim p_z(z)$ (e.g., Gaussian or uniform) to a synthetic sample $G(z)$. Its goal is to produce samples that the discriminator will classify as *real*.

Discriminator (D). The discriminator is a binary classifier that receives either a real sample $x \sim p_{data}(x)$ or a generated sample $G(z)$ and outputs $D(\cdot) \in (0, 1)$, the estimated probability that the input is real. The discriminator's aim is to correctly distinguish real from fake samples.

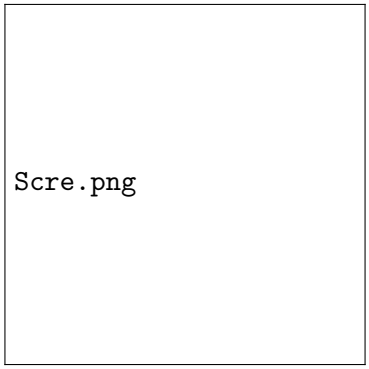

Figure 1: Typical architecture of a Generative Adversarial Network (GAN). The generator produces synthetic data from random noise, while the discriminator distinguishes real data from generated samples.

Loss Functions

GANs are trained via adversarial (minimax) optimization. The classic losses are:

Discriminator loss

$$J_D \;=\; -\frac{1}{m}\sum_{i=1}^{m}\big[\log D(x_i) + \log(1 - D(G(z_i)))\big],$$

where x_i are real samples and z_i are noise vectors.

Generator loss (non-saturating)

A commonly used practical generator objective is:

$$J_G \;=\; -\frac{1}{m}\sum_{i=1}^{m}\log D(G(z_i)).$$

This encourages the generator to increase the discriminator's belief that generated samples are real.

Minimax formulation

The original GAN formulation is:

$$\min_G \max_D V(G,D) = \mathbb{E}x \sim pdata[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))].$$

At equilibrium (idealized), the generator distribution p_G matches the true data distribution p_{data}.

Training Remarks and Challenges

Training GANs is delicate. Typical challenges include:

- **Mode collapse:** Generator produces limited variety of samples.
- **Instability:** Oscillations and failure to converge due to adversarial dynamics.
- **Evaluation difficulty:** Quantifying sample quality and diversity is nontrivial.

Many variants (Wasserstein GAN, Conditional GAN, Dual-Discriminator GAN, etc.) and architectural improvements (DCGAN, SAGAN) address these issues.

Mode Collapse: A GAN's Achilles' Heel

Mode Collapse

Mode collapse in Generative Adversarial Networks (GANs) occurs when the generator produces a limited range of outputs, failing to capture the full diversity of the true data distribution. It is analogous to an artist repeatedly creating the same style of artwork after achieving success, avoiding creative exploration. Similarly, a generator learns to reproduce only a few dominant data patterns that can easily deceive the discriminator, leading to repetitive outputs and loss of variety.

Understanding Mode Collapse

During GAN training, the generator and discriminator compete in a minimax game. When the discriminator becomes too strong or the optimization converges to a local minimum, the generator exploits only one or a few modes of the data distribution. For example, in a dataset with eight Gaussian clusters, the generator may initially produce samples from all clusters but eventually collapse to one. In image datasets such as MNIST, this can manifest as the generator producing only images of a single digit (e.g., "1"), ignoring others.

Mitigation Strategies

Several architectural and training strategies have been developed to alleviate mode collapse:

- **Wasserstein GANs (WGANs):** Replace the cross-entropy loss with the Wasserstein distance, providing smoother gradients and more stable learning. WGANs effectively reduce mode collapse by encouraging continuous improvement of both networks.
- **Unrolled GANs:** Incorporate future discriminator updates into the generator's loss, preventing the generator from over-optimizing against a single discriminator state and promoting diversity in outputs.
- **Regularization and noise:** Adding input or output noise, dropout, or weight decay helps maintain gradient variability and exploration.
- **Training heuristics:** Adjusting learning rates, balancing generator–discriminator capacity, or modifying network architectures can also mitigate collapse.

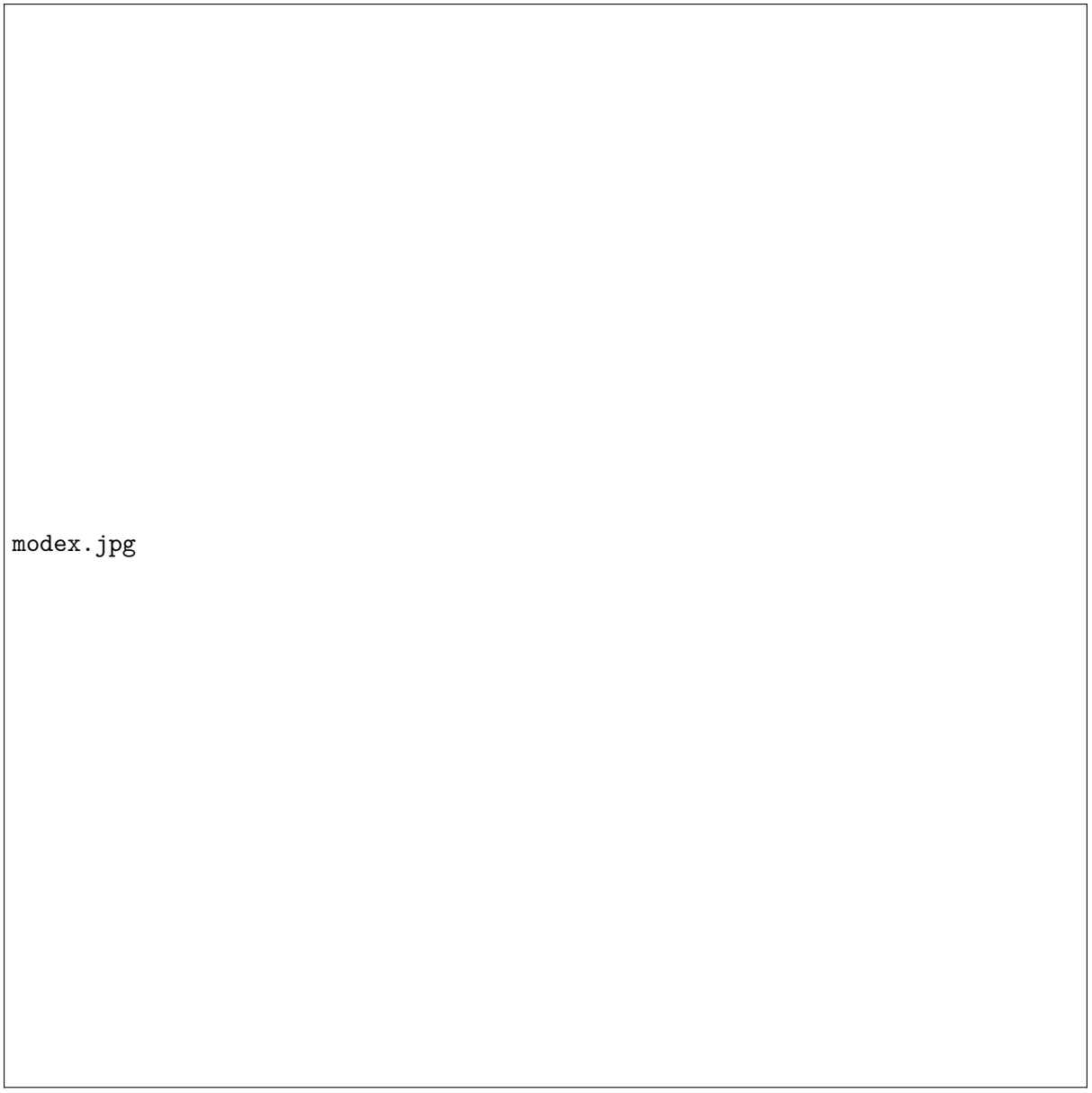

Figure 2: An illustration of the mode collapse problem on a two-dimensional toy dataset.

Deep Convolutional Generative Adversarial Networks

Introduction

The **Deep Convolutional Generative Adversarial Network (DCGAN)** is an extension of the original **GAN** framework designed to stabilize training and produce more realistic image outputs. It consists of two convolutional neural networks — a generator and a discriminator — that are trained simultaneously in an adversarial setup. The generator begins with a random noise vector and gradually upsamples it through a series of transposed convolutional layers to synthesize realistic images, learning hierarchical representations such as shapes, textures, and fine details. The discriminator, conversely, is a conventional convolutional network that takes an image as input and classifies it as real or generated, learning rich feature representations in the process.

Architecture and Working

The Deep Convolutional Generative Adversarial Network (DCGAN) follows a carefully designed architecture that promotes stable training and high-quality image generation. Its structure consists of two main components — a **generator** and a **discriminator** — both built entirely from convolutional layers. The following design guidelines are applied to achieve stability and efficiency:

- **Convolutional Design:** All pooling layers are removed and replaced with *strided convolutions* in the discriminator and *fractionally-strided (transposed) convolutions* in the generator. This allows the networks to learn their own spatial downsampling and upsampling operations.
- **Batch Normalization:** Batch normalization is applied in both the generator and the discriminator to stabilize learning by normalizing activations and improving gradient flow.
- **Fully Connected Layers:** Fully connected hidden layers are eliminated to enable deeper convolutional architectures that better capture hierarchical image features.
- **Activation Functions:** The generator uses **ReLU** activation in all layers except the output layer, which uses a **Tanh** activation to scale output pixels between -1 and 1. The discriminator, on the other hand, employs **LeakyReLU** activation in all layers to ensure smooth gradient propagation even for negative inputs.

- **Adversarial Learning Process:** The generator learns to produce realistic images from random noise vectors, while the discriminator learns to distinguish between real and generated images. Both networks are trained simultaneously in a minimax game, driving each other to improve progressively until the generated images become indistinguishable from real data.

These architectural principles collectively make DCGANs stable to train, enhance feature learning within convolutional layers, and lead to visually coherent image synthesis with meaningful latent representations.

Transposed Convolution

The process begins with a low-dimensional noise vector that serves as the input — this random vector captures different possible variations in the generated output. The generator first projects and reshapes this vector into a small, dense feature map, which forms the foundation for image construction. Through a series of transposed convolutional (fractionally-strided convolution) layers, the generator gradually increases the spatial dimensions of this feature map while reducing the number of feature channels. At each layer, the network learns to add structure and detail — from basic shapes and edges to more complex textures and patterns. As the data moves through the layers, it transitions from abstract latent features to visually meaningful image components. The final layer produces an output image with the desired dimensions (e.g., $64\times64\times3$ for an RGB image). Typically, ReLU activations are used in the intermediate layers for nonlinearity, and a Tanh activation is applied at the output to normalize the pixel values between -1 and 1.

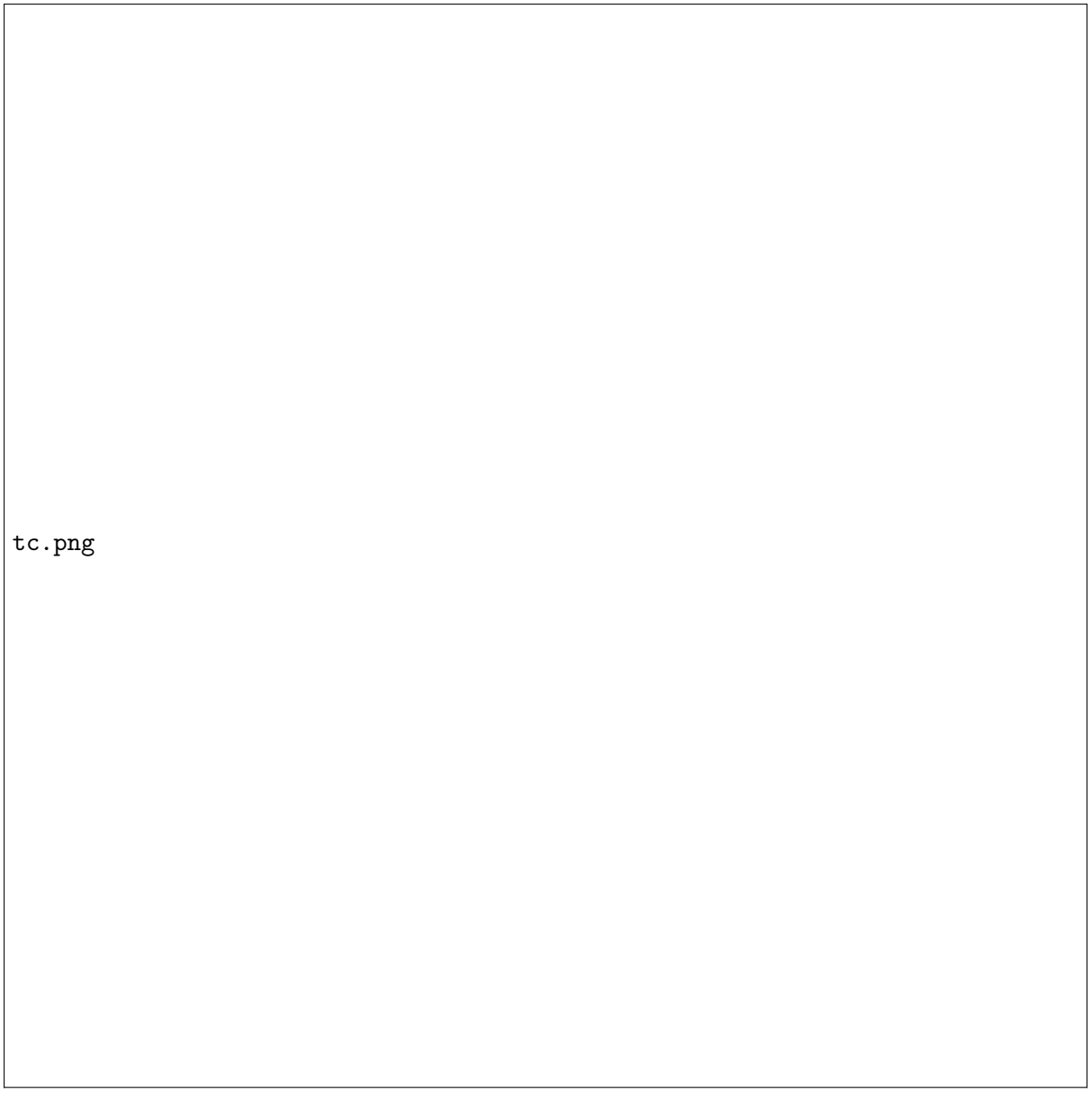

Figure 3: DCGAN generator used for LSUN scene modeling. A 100 dimensional uniform distribution Z is projected to a small spatial extent convolutional representation with many feature maps. A series of four fractionally-strided convolutions (in some recent papers, these are wrongly called deconvolutions) then convert this high level representation into a 64×64 pixel image. Notably, no fully connected or pooling layers are used.

Dual Discriminator Generative Adversarial Network (D2GAN)

Introduction

The **Dual Discriminator Generative Adversarial Network (D2GAN)** is an advanced variant of GANs proposed to address the problem of **mode collapse**, a common issue where the generator produces limited types of samples, failing to represent the full diversity of real data. Introduced by Nguyen et al. (NeurIPS 2017), D2GAN modifies the traditional two-player

GAN setup into a **three-player game** consisting of a generator and two discriminators. This framework effectively balances diversity and realism, enabling stable and scalable training on large datasets such as ImageNet.

Architecture

D2GAN consists of:

- **Generator (G):** Maps noise $z \sim p_z(z)$ to realistic data samples $G(z)$, aiming to fool both discriminators.
- **Discriminator 1 (D1):** Rewards high scores for real samples and penalizes generated ones, promoting full coverage of the data distribution.
- **Discriminator 2 (D2):** Rewards high scores for generated samples and penalizes real ones, encouraging realism and quality.

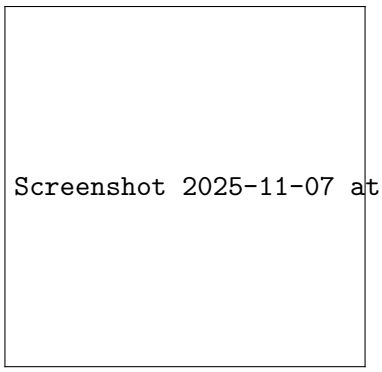

Figure 4: Figure 2: An illustration of the mode collapse problem on a two-dimensional toy dataset.

The training objective combines both discriminators as:

$$\min_{G} \max_{D_1,D_2} J(G, D_1, D_2) = \alpha\, \mathbb{E}x \sim pdata[\log D_1(x)] + \mathbb{E}_{z\sim p_z}[-D_1(G(z))] + \mathbb{E}x \sim pdata[-D_2(x)] + \beta\, \mathbb{E}_{z\sim p_z}[\log D_2(G$$

where α and β control the influence of the forward and reverse KL divergences.

Theoretical Analysis

For a fixed generator G, the optimal discriminators are:

$$D_1^*(x) = \frac{\alpha\, p_{data}(x)}{p_G(x)}, \quad D_2^*(x) = \frac{\beta\, p_G(x)}{p_{data}(x)}.$$

Substituting these into the objective yields:

$$J(G) = \alpha\, D_{KL}(p_{data} \| p_G) + \beta\, D_{KL}(p_G \| p_{data}) + \text{const.}$$

Thus, D2GAN minimizes both **forward** and **reverse** KL divergences, balancing diversity (mode coverage) and realism (sample quality). The equilibrium occurs when $p_G = p_{data}$, where both discriminators output constants ($D_1 = \alpha, D_2 = \beta$).

Self Attention

Introduction

Traditional **Generative Adversarial Networks (GANs)** primarily rely on convolutional layers to capture spatial dependencies within images. While effective for learning local features, convolutional operations have a limited receptive field and struggle to model long-range dependencies, often resulting in inconsistent textures or object structures in generated images. To address this limitation, Self-Attention mechanisms were introduced into GANs, allowing the model to capture relationships between distant spatial regions within an image. In a **Self-Attention GAN (SAGAN)**, each feature in the generator and discriminator can attend to all other features, enabling the network to focus on the most relevant parts of the image during synthesis. By combining convolutional processing for local feature extraction with self-attention for global context understanding, Self-Attention GANs achieve a more balanced and expressive representation of visual data.

Working

- The convolutional feature maps (denoted as x) extracted from earlier layers of the network are passed through three separate 1×1 convolutional transformations to produce f(x), g(x), and h(x) — which represent the query, key, and value feature maps, respectively.
- The f(x) and g(x) outputs are then reshaped and multiplied to measure the similarity between every pair of spatial positions, producing an attention map after applying a softmax function. This attention map indicates how strongly one pixel (or feature location) relates to another across the entire image.
- The h(x) features are then weighted by this attention map to form v(x), and the result is combined to generate the self-attention feature map (o). This output enhances each feature by incorporating global contextual information from all other spatial locations.
- In essence, this mechanism enables the network to focus on important regions and maintain global coherence in the generated images, ensuring that distant parts of the image are meaningfully related rather than independently generated.

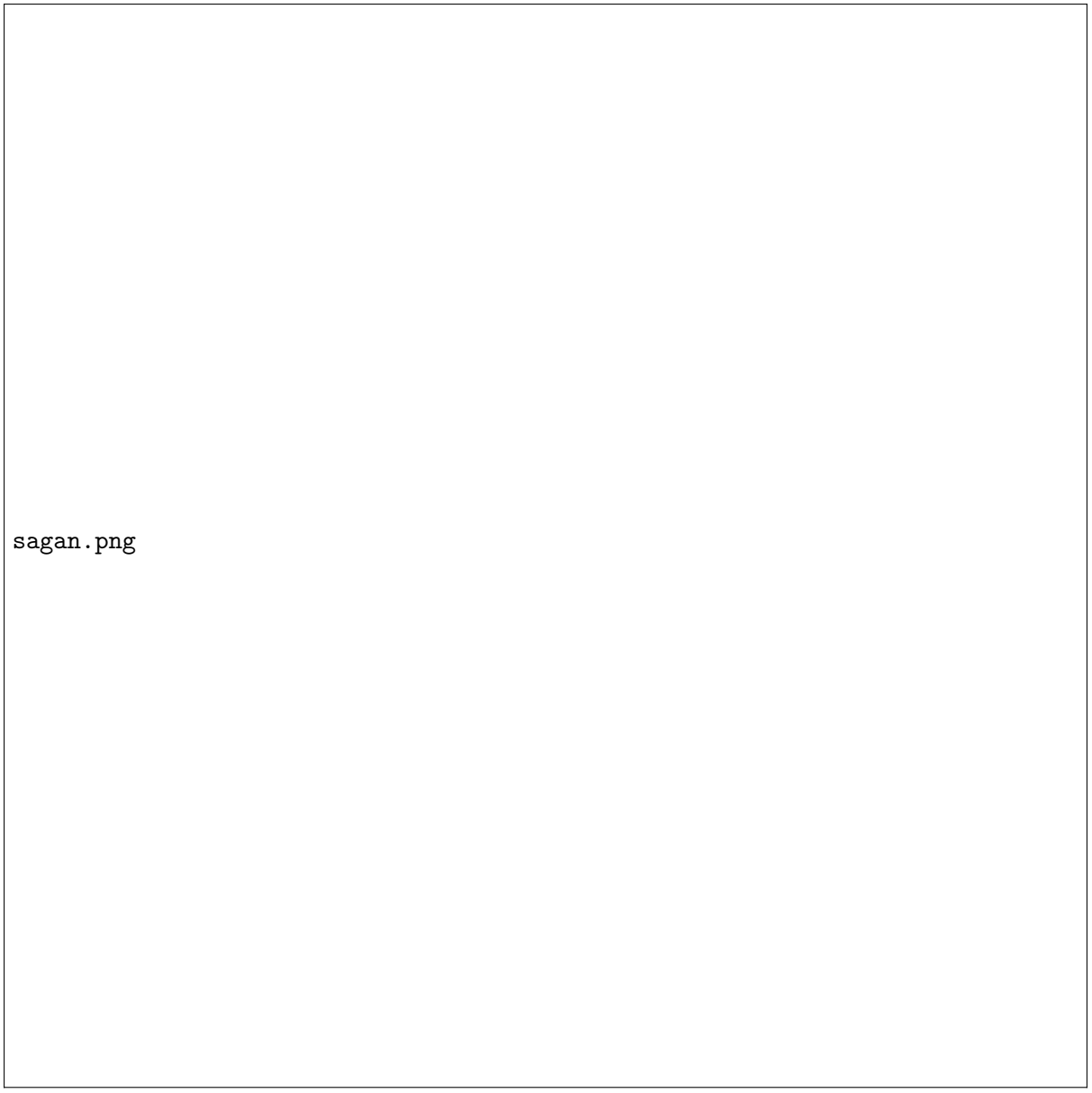

Figure 5: The proposed self-attention module for the SAGAN. The $\otimes$ denotes matrix multiplication. The softmax operation is performed on each row.

Literary Analysis

Traditional GANs rely on convolutional layers that effectively capture local patterns but struggle to model long-range dependencies. The Self-Attention GAN (SAGAN) addresses this limitation by introducing a self-attention mechanism that allows each spatial region of an image to relate to every other region. This enables the generator and discriminator to capture both local and global context, improving image coherence and feature consistency.

In SAGAN, the convolutional feature maps are transformed into three parallel representations: $f(x)$, $g(x)$, and $h(x)$, representing the query, key, and value mappings. The attention map is

computed using:

$$\beta_{ij} = \frac{\exp(f(x_i)^T g(x_j))}{\sum_{i=1}^{N} \exp(f(x_i)^T g(x_j))},$$

and the self-attended output is given by:

$$o_j = \sum_{i=1}^{N} \beta_{ij} h(x_i).$$

This mechanism allows the network to emphasize important spatial relationships and maintain global consistency. Additionally, spectral normalization is applied to stabilize training by controlling the magnitude of layer weights. Together, these enhancements make SAGAN more stable and capable of generating high-quality, globally coherent images compared to conventional GANs.

Table 1: D2GAN architecture and training hyperparameters used for CIFAR-10 (32×32). The generator upsamples a 100-D latent vector into a 32×32 RGB image. Both discriminators (D1 and D2) share identical architectures but optimize different objectives, encouraging diversity and realism simultaneously.

Network	**Layer**	**Output Size**
Generator	Input $z \sim \mathcal{N}(0, I)$	$100 \times 1 \times 1$
	ConvT($100 \to 512,\ 4 \times 4,\ s$=1) + BN + ReLU	$512 \times 4 \times 4$
	ConvT($512 \to 512,\ 3 \times 3,\ s$=1) + BN + ReLU	$512 \times 4 \times 4$
	ConvT($512 \to 256,\ 4 \times 4,\ s$=2) + BN + ReLU	$256 \times 8 \times 8$
	ConvT($256 \to 256,\ 3 \times 3,\ s$=1) + BN + ReLU	$256 \times 8 \times 8$
	ConvT($256 \to 128,\ 4 \times 4,\ s$=2) + BN + ReLU	$128 \times 16 \times 16$
	ConvT($128 \to 128,\ 3 \times 3,\ s$=1) + BN + ReLU	$128 \times 16 \times 16$
	ConvT($128 \to 3,\ 4 \times 4,\ s$=2) + Tanh	$3 \times 32 \times 32$
D1, D2 (shared)	Input image	$3 \times 32 \times 32$
	Conv($3 \to 128,\ 4 \times 4,\ s$=2) + LeakyReLU	$128 \times 16 \times 16$
	Conv($128 \to 256,\ 4 \times 4,\ s$=2) + BN + LeakyReLU	$256 \times 8 \times 8$
	Conv($256 \to 512,\ 4 \times 4,\ s$=2) + BN + LeakyReLU	$512 \times 4 \times 4$
	Conv($512 \to 1,\ 4 \times 4,\ s$=1) + Softplus	$1 \times 1 \times 1$
	Output score (positive real)	scalar
Training Params	Latent dimension z	100
	Batch size	64
	Optimizer	Adam
	Learning rate	2×10^{-4}
	Adam betas	$(\beta_1 = 0.5,\ \beta_2 = 0.999)$
	Number of epochs	100
	Weight initialization	Normal(0, 0.02)
	Dataset	CIFAR-10 (50k images)
	Image normalization	$[-1, 1]$ via mean=0.5, std=0.5

Table 2: D2GAN architecture and training hyperparameters used for CIFAR-10 (32×32). The generator upsamples a 100-D latent vector into a 32×32 RGB image. Both discriminators (D1 and D2) share identical architectures but optimize different objectives, encouraging diversity and realism simultaneously.

Network	**Layer**	**Output Size**
Generator	Input $z \sim \mathcal{N}(0, I)$	$100 \times 1 \times 1$
	ConvT($100 \to 512,\ 4 \times 4,\ s$=1) + BN + ReLU	$512 \times 4 \times 4$
	ConvT($512 \to 512,\ 3 \times 3,\ s$=1) + BN + ReLU	$512 \times 4 \times 4$
	ConvT($512 \to 256,\ 4 \times 4,\ s$=2) + BN + ReLU	$256 \times 8 \times 8$
	ConvT($256 \to 256,\ 3 \times 3,\ s$=1) + BN + ReLU	$256 \times 8 \times 8$
	ConvT($256 \to 128,\ 4 \times 4,\ s$=2) + BN + ReLU	$128 \times 16 \times 16$
	ConvT($128 \to 128,\ 3 \times 3,\ s$=1) + BN + ReLU	$128 \times 16 \times 16$
	ConvT($128 \to 3,\ 4 \times 4,\ s$=2) + Tanh	$3 \times 32 \times 32$
D1, D2 (shared)	Input image	$3 \times 32 \times 32$
	Conv($3 \to 128,\ 4 \times 4,\ s$=2) + LeakyReLU	$128 \times 16 \times 16$
	Conv($128 \to 256,\ 4 \times 4,\ s$=2) + BN + LeakyReLU	$256 \times 8 \times 8$
	Conv($256 \to 512,\ 4 \times 4,\ s$=2) + BN + LeakyReLU	$512 \times 4 \times 4$
	Conv($512 \to 1,\ 4 \times 4,\ s$=1) + Softplus	$1 \times 1 \times 1$
	Output score (positive real)	scalar
Training Params	Latent dimension z	100
	Batch size	64
	Optimizer	Adam
	Learning rate	2×10^{-4}
	Adam betas	$(\beta_1 = 0.5,\ \beta_2 = 0.999)$
	Number of epochs	50 (configurable)
	Weight initialization	Normal(0, 0.02)
	Dataset	CIFAR-10 (50k images)
	Image normalization	$[-1, 1]$ via mean=0.5, std=0.5